Lecture 2-3 — Regular Expressions in JavaScript

Introduction

Regular expressions (regex) are patterns used to match text. They are extremely useful for validation, searching, and simple parsing. In JavaScript, regex can be created using a literal (e.g. `/abc/`) or the `RegExp` constructor.

Quick syntax summary

- **Literal:** `/pattern/flags`
- **Constructor:** `new RegExp("pattern","flags")`
- **Flags:** `g` (global), `i` (case-insensitive), `m` (multiline)
- **Anchors:** `^` start, `$` end
- **Character classes:** `[abc]`, ranges `[a-z]`, negation `[^0-9]`
- **Predefined classes:** `\d` digits, `\w` word, `\s` whitespace
- **Quantifiers:** `*` (0 or more), `+` (1 or more), `?` (0 or 1), `{n,m}`
- **Groups:** `(...)`, alternation `|`

Example A — Basic tests and flags

```
// Using a regex literal
const r1 = /hello/i; // case-insensitive
console.log(r1.test('Hello')); // true

// Using constructor
const r2 = new RegExp('^\\d+$'); // matches only digits from start to end
console.log(r2.test('12345')); // true
console.log(r2.test('12a45')); // false
```

Output:

Explanation

test() returns true/false depending on whether the pattern matches. The `i` flag makes matching case-insensitive. Anchors `^` and `$` require the whole string to match when used together.

Example B — Common patterns

Here are some frequent needs and simple patterns for them.

```
// digits only:  ^\d+$
// basic email (simple, not perfect):  ^[\w.-]+@[\w.-]+\.[A-Za-z]{2,}$
// optional country code phone (very basic):  ^(\+\d{1,3}\s?)?\d{7,14}$
// PAN (from previous lecture):  /^[A-Z]{5}[0-9]{4}[A-Z]$/
```

Output:

Explanation (beginner-friendly)

- **^[...]$**: anchors force the whole string to match.
- **\d** means any digit (0–9). **\w** matches letters, digits and underscore.
- **+** is “one or more”. ***** is “zero or more”. **{n,m}** sets exact or range counts.
- The PAN pattern requires 5 uppercase letters, 4 digits, and 1 uppercase letter. It will fail for lowercase input.

Example C — Capturing groups and replacement

```
// Swap first and last name using groups
const name = 'Jane Doe';
const swapped = name.replace(/(\w+)\s+(\w+)/, '$2, $1'); // 'Doe, Jane'
console.log(swapped);
```

Output:

Explanation

Parentheses create groups. In the replacement string `$1`, `$2` refer to the text matched by the first and second group respectively.

Example D — Using `match` and global flag

```
const str = 'one two three two';
const found = str.match(/two/g); // returns array ['two','two']
console.log(found);
```

Output:

Example E — Lookahead and Lookbehind

```
// digits followed by 'px' (positive lookahead)
const sizes = '10px 5em 20px';
const px = sizes.match(/\d+(?=px)/g); // ['10','20']

// word preceded by $ (positive lookbehind)
const money = 'Total $50 and $100';
const amounts = money.match(/(?<=\$)\d+/g); // ['50','100']
```

Output:

Explanation

Lookahead `(?=...)` checks what comes after a match without including it. Lookbehind `(?<=...)` checks what comes before without including it. They are useful for extracting values that appear next to specific markers.

Example F — Greedy vs Lazy quantifiers and a simple URL test

```
// Greedy vs lazy
const html = '
one

two
';
const greedy = html.match(/
.*<\/div>/);   // matches entire string
const lazy = html.match(/
.*?<\/div>/);    // matches first div only

// Simple URL test (basic)
```

```
const url = /^https?:\/\/[\w.-]+(\.[\w.-]+)+\/?$/i;
console.log(url.test('https://example.com'));
```

Output:

Explanation

Greedy quantifiers try to match as much as possible. Lazy quantifiers (add `?`) match the smallest possible amount. Use lazy quantifiers when you want the shortest match.

Example G — Splitting and replace with callback

```
// split by comma
const list = 'alice,bob,charlie';
const parts = list.split(','); // ['alice','bob','charlie']

// replace with callback to format numbers
const prices = '10 20 30';
const formatted = prices.replace(/\d+/g, (m) => '$' + m + '.00'); // '$10.00 $20.00 $30.00'
```

Output:

Good practices & tips

- Start simple: build and test small parts of a pattern.
- Use online regex testers when learning (Regex101, RegExr) but remember JavaScript flavor differences.
- Avoid over-complicated single-line regexes for important validations (like full email validation).

- Escape special characters when matching them literally (e.g. `\.` for a literal dot).

Summary

Regular expressions let you describe text patterns concisely. Use literals for simple patterns and the `RegExp` constructor when you need dynamic patterns. Practice common building blocks: character classes, quantifiers, anchors, groups, and flags.

End of Lecture 2-3